House Price Prediction

Machine Learning Project — Interview Preparation Guide

Project stack: California Housing dataset, Pandas, scikit-learn, Bagging Regressor, Gradient Boosting Regressor, Joblib, Streamlit, Matplotlib and Seaborn.

Project workflow: Load data → separate features and target → split into training and testing sets → train both models in separate Python scripts → evaluate using MSE and R^2 → save model files and CSV results → load saved models in Streamlit → display comparisons and predict a new house value.

How to use this guide

Practise explaining the project aloud. Remember the settings from your code, understand the concepts, and use your actual printed MSE and R^2 results if asked. Do not invent performance numbers.

1. Project Introduction

Q1. Tell me about your project.

My project is House Price Prediction using Ensemble Learning. I used the California Housing dataset to predict house values with two algorithms: Bagging Regressor and Gradient Boosting Regressor. I loaded the data using Pandas, separated the input features and target, and split the data into 80% training and 20% testing sets. For Bagging, I used Decision Tree Regressors with 10 estimators. For Gradient Boosting, I used 100 estimators, a learning rate of 0.1 and a maximum tree depth of 3. I evaluated both models using Mean Squared Error and R^2. I saved the trained models using Joblib and saved metrics and predictions as CSV files. Finally, I built a Streamlit dashboard to compare model performance, visualize actual versus predicted values, and predict a new house value using a selected saved model.

Q2. Why did you choose this project?

I chose this project to practise regression and ensemble learning, compare different models, and build a usable dashboard for predictions.

Q3. What is the objective?

To estimate house values from housing-related features and compare the performance of Bagging and Gradient Boosting.

Q4. Which dataset did you use?

The California Housing dataset. It includes district-level information such as median income, house age, average rooms, population, occupancy, latitude, longitude, and a target house value.

Q5. What are the input features?

MedInc (median income), HouseAge (house age), AveRooms (average rooms), AveBedrms (average bedrooms), Population, AveOccup (average occupancy), Latitude and Longitude. The target column is what the model predicts.

Q6. Is this classification or regression?

Regression, because the model predicts a continuous numerical value rather than a class label.

Q7. Why split the data?

To train on one portion and evaluate on held-out observations, helping estimate how well the model generalizes to unseen data.

Q8. What is Ensemble Learning?

A machine-learning approach that combines multiple models to improve predictive performance, stability, or accuracy.

Q9. Why use Bagging and Boosting?

They are two different ensemble approaches. Bagging combines independently trained models; Boosting builds models sequentially to improve the current ensemble.

Q10. Why use Decision Trees as the Bagging base estimator?

Decision Trees can model nonlinear relationships and interactions. Bagging trains multiple trees on bootstrap samples and averages their predictions, which can reduce variance.

Q11. What is a bootstrap sample?

A sample drawn randomly from the training data with replacement. An observation can appear more than once in a sample.

Q12. What is n_estimators?

It specifies the number of individual estimators or trees in the ensemble. This project uses 10 for Bagging and 100 for Gradient Boosting.

Q13. What is learning_rate?

It controls how much each new tree contributes to the boosting ensemble. A smaller value makes learning more gradual and may require more trees. This project uses 0.1.

Q14. What does max_depth=3 mean?

It limits each boosting tree to a maximum depth of 3, helping control tree complexity.

Q15. Why random_state=42?

It makes randomized operations reproducible when the data and settings are unchanged.

2. Evaluation Metrics

Q16. What is Mean Squared Error (MSE)?

MSE is the average of squared differences between actual and predicted values. Lower MSE is generally better on the same evaluation data.

Formula

MSE = (1/n) × sum((actual value − predicted value)²).

Q17. What is R²?

R² measures how well a regression model explains variation in target values compared with predicting the mean. R² = 1 indicates perfect predictions; R² = 0 corresponds to the mean-prediction baseline under the standard definition; a negative score is worse than that baseline.

Q18. How do you choose the better model?

Compare the test-set MSE and R². Generally, lower MSE and higher R² are preferred, provided the model performs reliably on unseen data. Use the actual results printed by your scripts.

3. Python and Implementation

Q19. Why Pandas?

To load the CSV, manipulate the dataset, prepare results tables, and save metrics and predictions.

Q20. Explain X = df.drop('target', axis=1) and Y = df['target'].

X contains all columns except target and is used as input. Y contains the target values the model learns to predict. axis=1 means a column is being dropped.

Q21. Explain train_test_split(..., test_size=0.2, random_state=42).

It reserves 20% of the observations for testing and uses 80% for training. random_state makes the split reproducible.

Q22. What are fit() and predict()?

fit() trains the model on training data. predict() uses the trained model to estimate values for supplied features.

Q23. What is Joblib?

Joblib serializes Python objects such as scikit-learn models. joblib.dump() saves a model and joblib.load() loads it for prediction without retraining.

Q24. Why save CSV files?

Result CSVs store model metrics; prediction CSVs store actual and predicted test values. The dashboard can display these without rerunning training.

Q25. Why Streamlit?

It creates interactive web applications in Python. The dashboard displays metrics, comparison plots, prediction tables, and a form for new house estimates.

Q26. Does the dashboard train the models?

No. The separate Python scripts train and save the models. The dashboard loads the saved model files and uses them for predictions.

Q27. Why Matplotlib and Seaborn?

Matplotlib creates and customizes figures; Seaborn makes statistical plots such as model-comparison bar charts easier to build.

4. Advanced Questions

Q28. Can Bagging overfit?

Yes. Bagging often reduces variance for unstable models such as Decision Trees, but it does not guarantee that overfitting disappears.

Q29. Can Boosting overfit?

Yes. Excessive model complexity or too many trees can overfit. Tree depth, learning rate, number of estimators, and validation-based early stopping can help control this.

Q30. Why not Linear Regression?

This project focuses on comparing ensemble methods. Linear Regression could be added as a baseline to see whether the more complex models improve performance.

Q31. Why no feature scaling?

Tree-based models such as Decision Trees and Gradient Boosting generally do not require feature scaling because their splits depend on feature thresholds rather than comparable feature magnitudes.

Q32. What if a user enters incorrect house details?

The model may still produce a prediction, but it can be unreliable. Input validation and checking values against the training-data distribution would improve the app.

Q33. Is the result a real market price?

No. It is an estimate learned from the California Housing dataset, not a live property valuation. Its usefulness depends on how representative the data is for the property.

Q34. What would you improve?

I would add a Linear Regression baseline, tune hyperparameters, test more models, validate input ranges, and improve prediction explanations.

5. Dashboard Questions

Q35. What does Actual vs Predicted show?

It compares actual values with predictions. Points closer to the diagonal line represent predictions closer to the actual values.

Q36. Why multiply the prediction by 100,000?

The standard California Housing target is in units of $100,000. If your CSV retains these original units, multiplying the predicted target by 100,000 converts it to dollars.

Q37. Difference between result CSV and prediction CSV?

The results CSV stores model name and evaluation metrics. The prediction CSV stores actual and predicted values for test observations. The .pkl file stores the trained model for future predictions.

6. Bagging vs Boosting — Quick Revision

Feature	Bagging	Boosting
Training	Models trained independently	Models trained sequentially
Main idea	Train on bootstrap samples and aggregate	New models improve the current ensemble
Typical benefit	Often reduces variance	Can reduce bias and improve fit
Project model	Bagging Regressor with Decision Trees	GradientBoostingRegressor

Short interview answer: Bagging trains multiple models independently on bootstrap samples and combines their predictions to reduce variance. Boosting trains models sequentially, with each new model improving the current ensemble by focusing on its errors.

7. Final Revision Checklist

■ Explain the project workflow without reading.

■ Know the eight input features and the target column.

■ Remember the 80:20 split, 10 Bagging estimators, and 100 Boosting estimators.

■ Explain MSE and R^2, including which direction is better.

■ Know why the dashboard loads .pkl models rather than retraining.

■ Be ready to state your actual MSE and R^2 values from your terminal.

■ Explain that predictions are estimates, not guaranteed current market prices.

Important: The dashboard's feature column names and target-unit conversion must match the actual CSV used to train the models. Only claim model performance values you have measured.